

Semiconductor Pass/Fail Prediction Using an Off-the-shelf Model

This was my Module 4 project for EAI 6010: Applications of Artificial Intelligence. I used the UCI SECOM dataset and turned an off-the-shelf AI idea into a cleaner tabular classification project. My focus was not only model score. I also cared about data audit, leakage-safe preprocessing, model comparison, and a more realistic quality-screening story.

Python

scikit-learn

PyTorch

Random Forest

SECOM

SIMPLE NOTE

This is my shorter portfolio version rewritten from my notebook, report, and slides. I kept the main logic, charts, and final results, but simplified the writing for faster reading.

BUSINESS PROBLEM

Can semiconductor sensor readings help flag likely fail units earlier? My goal was to build an honest screening model for quality monitoring, not to claim a full production-ready auto decision system.

PROJECT INFO

At a glance

Course: EAI 6010 - Applications of Artificial Intelligence

Term: Winter 2025

Type: Individual course project

Team: N/A

Original deliverables: Notebook, report, and slides

Dataset: UCI SECOM Dataset

Rows / columns: 1,567 rows / 590 sensor features

Main tools: pandas, scikit-learn, matplotlib, seaborn, PyTorch

Model note: Random Forest was selected on validation balanced accuracy with threshold tuning

Project Snapshot

The original assignment asked me to use an off-the-shelf model idea and adapt it to a new problem. I chose a semiconductor manufacturing dataset and rebuilt the workflow around tabular sensor data, class imbalance, missing values, and more careful evaluation.

BUSINESS FOCUS

Quality screening

Use sensor data to support earlier fail detection in a manufacturing setting.

DATA SCALE

1,567 x 590

Small dataset, many sensors, high missingness, and only 104 fail cases.

METHODS USED

EDA + model comparison

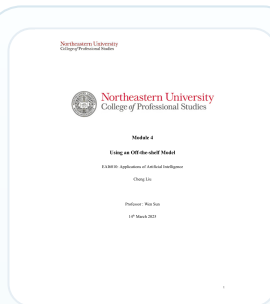
Compared Dummy, Logistic Regression + PCA, Random Forest, and a weighted MLP.

ORIGINAL DELIVERABLES

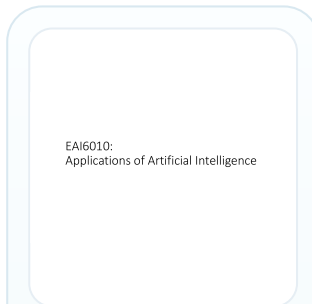
Notebook + report + slides

This PDF is the shorter visual version for portfolio use.

Original source materials consolidated here



Original report



Original slide deck

I reused the main story from my notebook and report, then rebuilt it into a shorter format that is easier for HR or an interviewer to scan.

My role in the project

- Picked the SECOM dataset and reframed the assignment around semiconductor pass/fail prediction.
- Built the notebook workflow for data loading, audit, preprocessing, and model comparison.
- Compared baseline models and tuned the decision threshold on the validation set.
- Wrote the final interpretation around deployment limits, drift risk, and manufacturing relevance.

Contribution note

This was an individual project. I owned the full workflow and this portfolio rewrite.

Problem, Dataset, and Analysis Plan

I treated this as a manufacturing quality-screening task: can sensor readings help flag likely fail units earlier? Because the dataset is small, imbalanced, and noisy, the project needed both modeling work and careful validation logic.

PROJECT PROPOSAL

Adapt an off-the-shelf AI workflow to a real tabular semiconductor dataset. Build a cleaner baseline pipeline, compare models, and explain whether the result would be safe to use in practice.

Dataset summary

Details

Main table	SECOM sensor matrix with 1,567 rows and 590 numeric features
Second table	Label file with pass/fail status and timestamp fields
Merge key	Row order alignment from the original UCI source files
Working data	Final classification dataset after load: 1,567 rows
Missing values	538 features have some missing values; 28 exceed 50% missing in the full audit

STAKEHOLDERS

Quality engineers, process engineers, manufacturing analysts, and operations or yield managers.

MAIN QUESTIONS

What does the data look like? Which model is most reliable on this imbalanced pass/fail problem? Does threshold tuning change the trade-off? Would I trust the result in a real fab workflow?

MY WORKFLOW

Load - audit - visualize - split - training-only preprocessing - compare models - tune threshold - holdout test - business interpretation.

SELECTED CODE - MAIN SETUP

```
# Load sensor data and labels
X_raw = pd.read_csv(data_file, sep=r"\s+", header=None)
labels_df = pd.read_csv(labels_file, sep=r"\s+",
header=None)

# Convert labels: -1 = pass -> 0, 1 = fail -> 1
y = labels_df["label"].replace({-1: 0, 1: 1}).astype(int)

# Stratified split
X_train_full, X_test, y_train_full, y_test =
train_test_split(
    X_raw, y, test_size=0.20, stratify=y, random_state=42
)
X_train, X_val, y_train, y_val = train_test_split(
    X_train_full, y_train_full, test_size=0.25,
    stratify=y_train_full, random_state=42
)
```

I first built a clean load and split step because I wanted the later preprocessing to stay leakage-safe.

LAB / CODE NOTE

I used a Python notebook in Colab. The main libraries were pandas, scikit-learn, matplotlib, seaborn, and PyTorch.

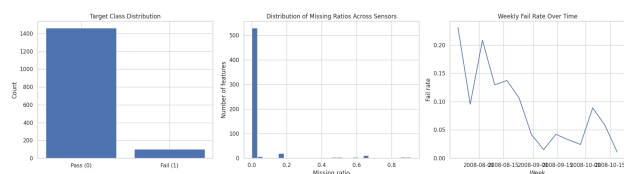
EXAMPLE FIELDS I USED

sensor_001, sensor_034, sensor_060, sensor_104
sensor_126, sensor_181, sensor_214, sensor_478
label, timestamp, missing_ratio, feature_importance

Key Findings and Visual Evidence

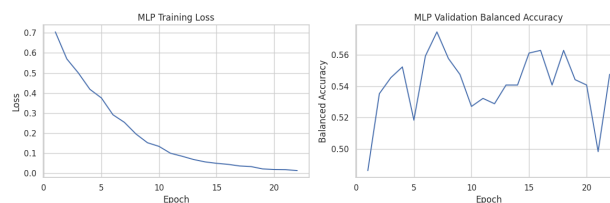
The first pass through the data already showed why this project was hard. The fail class was rare, many sensors had missing values, and the weekly fail rate moved over time.

Class balance, missingness, and weekly fail rate



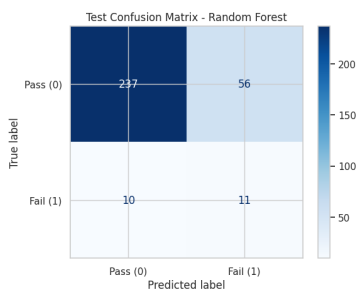
Suggested insert: use this chart set early. It shows the class imbalance, the spread of missing ratios, and why timestamps may matter for future validation design.

MLP learning check



I also tested a weighted MLP to stay aligned with the AI course. It learned, but it was not the most balanced final model for this dataset.

Final test confusion matrix



With the selected threshold, the Random Forest model caught 11 of 21 fail cases on the test set, but it also produced many false positives.

WHAT THESE CHARTS TOLD ME

- The fail class is only 6.64%, so accuracy alone can be very misleading.
- Missingness is spread across many sensors, so filtering and imputation matter a lot.
- The time plot suggests the process may not be fully stationary, so time-based validation is a good next step.
- The final model looks more useful as a screening tool than a fully automated accept/reject system.

Quick business read

This project is strongest as a practical baseline for quality screening, not as a production-ready classifier.

Methods, Model Comparison, and Final Choice

I kept the final workflow conservative. I first compared simple baselines, then selected the best validation candidate and tested it once on the untouched holdout set.

SELECTED CODE SNIPPET

```
# Training-only missingness filter
keep_cols = X_train.columns[X_train.isna().mean() <= 0.50]

# Median imputation for the tree path
imputer = SimpleImputer(strategy="median")
X_train_tree = imputer.fit_transform(X_train[keep_cols])
X_val_tree = imputer.transform(X_val[keep_cols])

# Threshold search on validation set
def find_best_threshold(y_true, y_prob):
    thresholds = np.linspace(0.05, 0.95, 181)
    return max(
        ((t, balanced_accuracy_score(y_true, (y_prob >=
t).astype(int))))
        for t in thresholds),
        key=lambda x: x[1]
    )
```

This was the key logic: fit preprocessing on training only, then tune the decision threshold on the validation split.

INTERPRETATION

- I split the data before imputation, scaling, or PCA to avoid leakage.
- I dropped features with more than 50% missing values using training data only.
- I used median imputation and kept PCA only for the linear path.
- I compared Dummy, Logistic Regression + PCA, Random Forest, and a weighted MLP.
- I tuned the decision threshold on the validation set instead of assuming 0.50 was always best.

Validation model comparison and final holdout result

Model	Split	Threshold	Balanced Acc.	Recall	PR-AUC
Random Forest	Validation	0.110	0.6458	0.5714	0.1329
Logistic Regression + PCA	Validation	0.305	0.6205	0.3810	0.1509
Dummy Classifier	Validation	0.500	0.5000	0.0000	0.0669
Random Forest	Test	0.110	0.6663	0.5238	0.2192

Random Forest won on validation balanced accuracy. Logistic Regression stayed competitive on some metrics, which helped me explain the trade-off instead of pretending one model was perfect.

Conclusion, Recommendations, and Interview Use

This project taught me how to turn a tutorial-style modeling idea into a cleaner applied AI workflow. The value was not just the final metric. It was the full reasoning around data quality, validation, threshold choice, and deployment risk.

MAIN FINDINGS

Class imbalance made plain accuracy unreliable.

Random Forest gave the best balanced validation result and a decent test ROC-AUC around 0.80.

RESULT MEANING

The model can support manual review or early screening, but it still creates too many false positives for full automation.

RECOMMENDATIONS

Add time-based validation, repeated resampling, cost-based threshold setting with engineers, and drift monitoring with retraining plans.

WHAT THIS PROJECT SHOWS ABOUT MY WORK

- I can audit messy, high-dimensional data before modeling.
- I can build a leakage-safe preprocessing and evaluation pipeline.
- I can compare models honestly instead of over-claiming one result.
- I can explain technical results in a simple business-facing way.

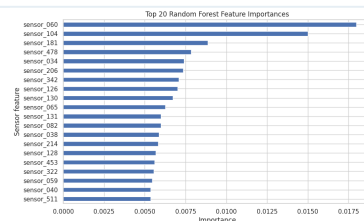
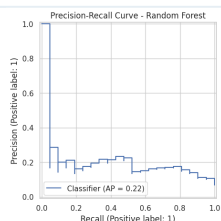
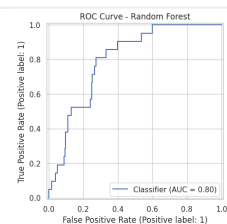
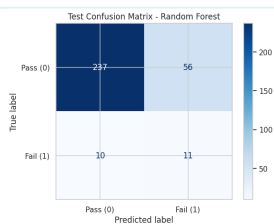
HOW I WOULD EXPLAIN THIS IN AN INTERVIEW

Simple version: I used the UCI SECOM dataset to build a semiconductor pass/fail screening model. I compared several approaches, handled missing values carefully, and picked Random Forest because it gave the best balanced validation result. The final model showed useful signal, but I would still position it as a prototype because of false positives, small fail counts, and possible time drift.

Final note

I kept this version short and visual so an interviewer can understand the project quickly. The full notebook still has the detailed workflow.

Quick Visual Gallery



Suggested insert: keep a small gallery at the end so readers can quickly revisit the core evidence - confusion matrix, ROC curve, PR curve, and feature importance.